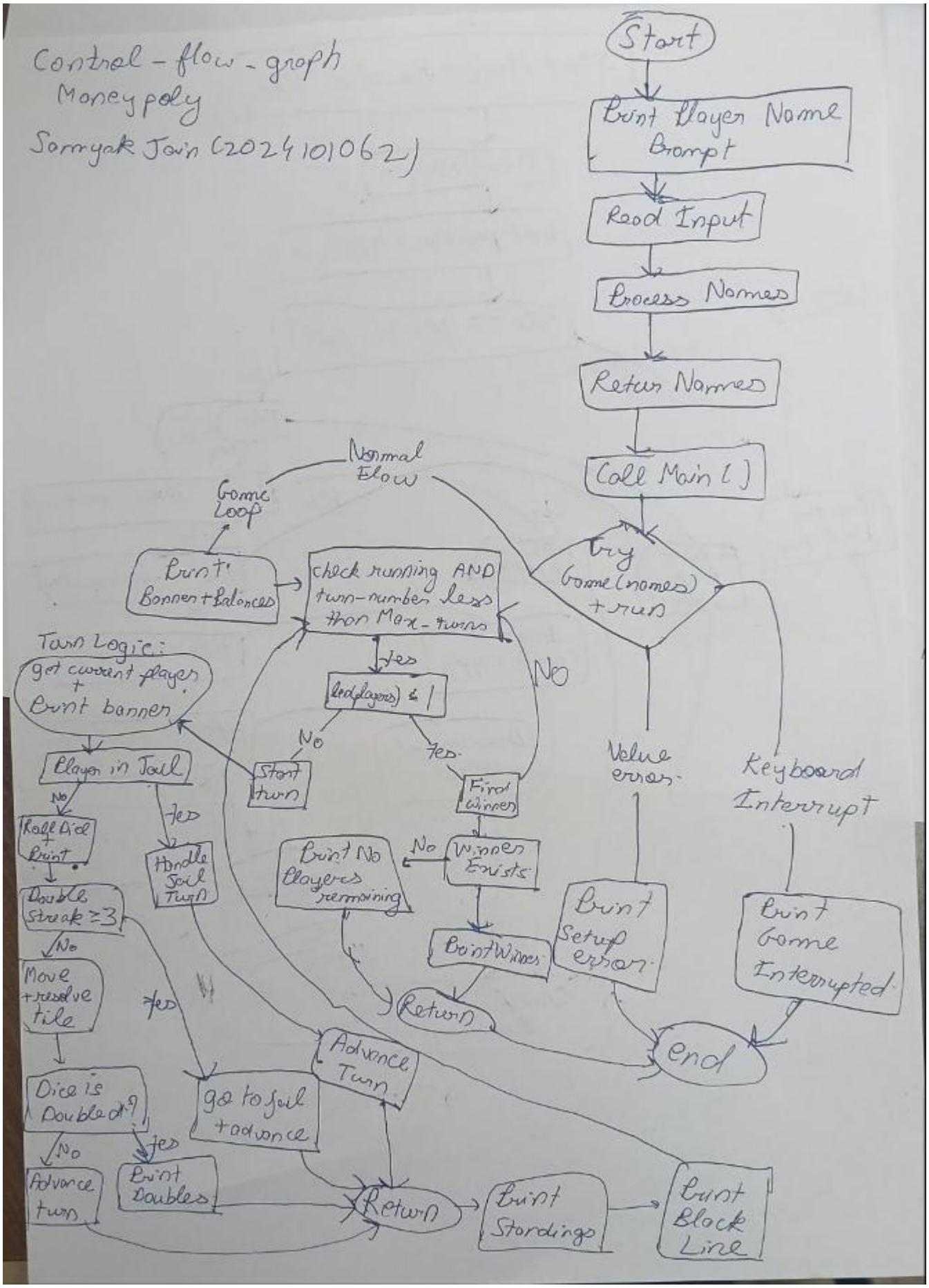
WHITE BOX TESTING REPORT

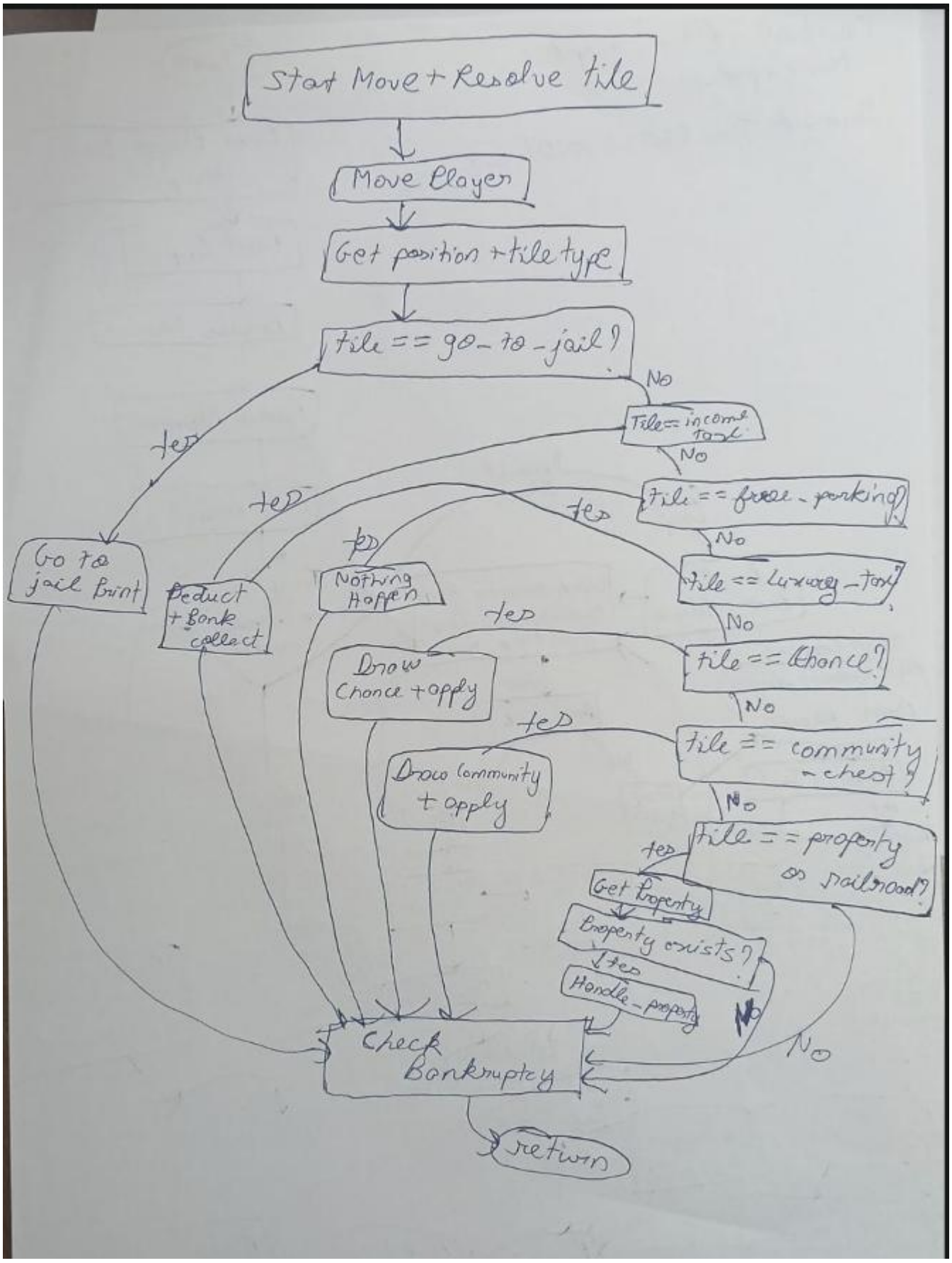
SamyakJain

2026-03-23

CONTROL FLOW GRAPH(BELOW)

Control - flow - graph
Moneypoly
Samyak Jain (2024101062)




Start Move + Resolve tile
Move Player
Get position + tile type
tile == go_to_jail?
Yes
No
Go to jail Print
Tile == income tax
Yes
No
Deduct + Bank collect
tile == free_parking?
Yes
No
Nothing Happen
tile == luxury_tax?
Yes
No
Draw Chance + apply
tile == Chance?
Yes
No
Draw community + apply
tile == community chest?
Yes
No
tile == property or railroad?
Yes
Get Property
Property exists?
Yes
No
Handle_property
Check Bankruptcy
No
return

1 Iteration Report (Included from Part 1.2)

Iteration 0

- Pushed source code and project structure to GitHub.

Iteration 1

- **Pylint score before:** 9.08/10
- **Pylint score after:** 9.23/10
- **Key fixes:** removed unused imports, simplified boolean comparison, fixed a non-interpolated f-string, cleaned conditions.

Iteration 2

- **Pylint score before:** 9.23/10
- **Pylint score after:** 9.31/10
- **Key fixes:** added module and function docstrings in main.py; added module/class docstrings in bank.py.

Iteration 3

- **Pylint score before:** 9.31/10
- **Pylint score after:** 9.39/10
- **Key fixes:** added module-level docstrings in board.py, dice.py, config.py, game.py, and player.py.

Iteration 4

- **Pylint score before:** 9.39/10
- **Pylint score after:** 9.77/10
- **Key fixes:** added cards.py module docstring; reformatted card dictionaries to satisfy line length limits.

Iteration 5

- **Pylint score before:** 9.77/10
- **Pylint score after:** 9.88/10
- **Key fixes:** initialized Dice.doubles streak in __init__, removed dead code in Player.move, improved property and UI exception handling.

Iteration 6

- **Pylint score before:** 9.88/10
- **Pylint score after:** 9.91/10
- **Key fixes:** simplified Game.interactive menu control flow; fixed final newline style issue in player.py.

Iteration 7

- **Pylint score before:** 9.91/10
- **Pylint score after:** 9.92/10
- **Key fixes:** reduced Property stored attributes; improved Board property creation readability.

Iteration 8

- **Pylint score before:** 9.92/10
- **Pylint score after:** 9.96/10
- **Key fixes:** introduced PropertySpec dataclass; updated Board property creation to use PropertySpec.

Iteration 9

- **Pylint score before:** 9.96/10
- **Pylint score after:** 9.97/10
- **Key fixes:** consolidated Player state into one internal dictionary with accessors and added accessor docstrings.

Iteration 10

- **Pylint score before:** 9.97/10
- **Pylint score after:** 10.00/10
- **Key fixes:** consolidated Game turn state and refactored tile/card handling to reduce branch complexity.

Iteration outcome

- Progressive quality improvements increased pylint from 9.08/10 to 10.00/10 while preserving game behavior.

2 PART 1.3

System Under Test: moneypoly (Part-1 WhiteBoxTesting)

Goal

- Cover all branches (if/elif/else paths), key state transitions, and edge cases.
- Explain in simple words why each test case is needed.
- Record all logical errors found by tests and the fixes applied.

Test Execution Summary

- **Total tests:** 190 across 12 files in Part_1.3
- **Latest run:** pytest -q Part_1.3
- **Result:** 190 passed

How to run all tests (pytest):
python -m pytest -q Part_1.3

How to run all tests (unittest discovery):
python -m unittest discover -s Part_1.3 -p "test_*_whitebox.py"

---

3 Defects Found and Corrected (Error Log, 20 total)

Error #1

- **Failing test(s):** test_cards_remaining_empty_expected_zero
- **Module:** moneypoly/cards.py
- **Root cause:** CardDeck.cards_remaining() used modulo with len(self.cards) even when the deck was empty, causing ZeroDivisionError.
- **Fix applied:** Added an empty-deck guard in cards_remaining() that returns 0 when self.cards is empty.
- **Commit message used:** Error 1: Fix CardDeck empty-deck cards_remaining ZeroDivisionError

Error #2

- **Failing test(s):** test_roll_calls_randint_with_full_six_sided_range
- **Module:** moneypoly/dice.py
- **Root cause:** Dice.roll() used random.randint(1, 5) for both dice, excluding face value 6.
- **Fix applied:** Updated Dice.roll() to use random.randint(1, 6) for both die1 and die2.
- **Commit message used:** Error 2: Fix Dice.roll range to use 1-6 for both dice

Error #3

- **Failing test(s):** test_find_winner_should_pick_richest
- **Module:** moneypoly/game.py
- **Root cause:** Game.find_winner() used min(...) with net worth, which selected the poorest player instead of the richest.
- **Fix applied:** Replaced min(...) with max(...) in find_winner() to return the player with highest net worth.
- **Commit message used:** Error 3: Fix find_winner to select highest net worth player

Error #4

- **Failing test(s):** test_player_move_passing_go_collects_salary
- **Module:** moneypoly/player.py
- **Root cause:** Player.move() only awards Go salary when landing exactly on 0, not when passing Go.
- **Fix applied:** Updated Player.move() to track old_position and award GO salary when forward movement crosses or lands on Go.
- **Commit message used:** Error 4: Fix Player.move to award GO salary when passing Go

Error #5

- **Failing test(s):** test_bank_collect_negative_amount_is_ignored
- **Module:** moneypoly/bank.py
- **Root cause:** Bank.collect() docstring says negative amounts are ignored, but implementation subtracts from bank funds.
- **Fix applied:** Added an early return in Bank.collect() when amount is negative so both bank funds and collection totals remain unchanged.
- **Commit message used:** Error 5: Ignore negative amounts in Bank.collect

Error #6

- **Failing test(s):** test_bank_give_loan_reduces_bank_funds
- **Module:** moneypoly/bank.py
- **Root cause:** Bank.give_loan() docstring says loan reduces bank funds, but implementation does not reduce_funds.
- **Fix applied:** Updated Bank.give_loan() to deduct loan amount from bank funds before crediting the player, and added a guard to prevent issuing loans larger than available bank reserves.
- **Commit message used:** Error 6: Deduct bank funds when issuing emergency loans

Error #7

- **Failing test(s):** test_buy_property_with_exact_balance_should_succeed
- **Module:** moneypoly/game.py
- **Root cause:** Game.buy_property() rejects purchase when balance == price due to ≤ check.
- **Fix applied:** Changed affordability check in Game.buy_property() from ≤ to < so exact-price purchases are allowed.
- **Commit message used:** Error 7: Allow buying property with exact balance

Error #8

- **Failing test(s):** test_pay_rent_transfers_money_to_owner
- **Module:** moneypoly/game.py
- **Root cause:** Game.pay_rent() deducts rent from tenant but does not credit the property owner.
- **Fix applied:** Updated Game.pay_rent() to add collected rent to prop.owner after deducting it from the tenant.
- **Commit message used:** Error 8: Transfer rent payment to property owner

Error #9

- **Failing test(s):** test_jail_pay_fine_should_deduct_player_balance
- **Module:** moneypoly/game.py
- **Root cause:** Voluntary jail-fine path credits bank but does not deduct fine from the player.
- **Fix applied:** Updated _handle_jail_turn() voluntary fine path to deduct JAIL_FINE from the player before collecting it into the bank.
- **Commit message used:** Error 9: Deduct player balance on voluntary jail fine payment

Error #10

- **Failing test(s):** test_trade_should_credit_seller_cash
- **Module:** moneypoly/game.py
- **Root cause:** Game.trade() deducts buyer cash but does not add that cash to seller balance.
- **Fix applied:** Updated Game.trade() to credit seller.add_money(cash_amount) when a trade succeeds.
- **Commit message used:** Error 10: Credit seller cash during property trade

Error #11

- **Failing test(s):** multiple Property construction tests (Property.__init__ signature mismatch)
- **Module:** moneypoly/property.py
- **Root cause:** Property.__init__ was refactored to accept only PropertySpec, breaking tests that pass legacy parameters.
- **Fix applied:** Added backward-compatible Property.__init__ that accepts either PropertySpec or legacy args.
- **Commit message used:** Error 11: Restore Property constructor compatibility

Error #12

- **Failing test(s):** test_move_resolve_chance_draws_card, test_move_resolve_community_draws_card
- **Module:** moneypoly/game.py
- **Root cause:** Game refactor removed chance_deck/community_deck attributes used by tests.
- **Fix applied:** Added chance_deck and community_deck accessors that expose decks.
- **Commit message used:** Error 12: Restore card deck accessors

Error #13

- **Failing test(s):** test_carddeck_repr_safe_when_empty
- **Module:** moneypoly/cards.py
- **Root cause:** CardDeck.__repr__ used modulo with len(self.cards) and crashed on empty deck.
- **Fix applied:** Added empty-deck guard returning "CardDeck(0 cards, next=none)".
- **Commit message used:** Error 13: Make CardDeck.__repr__ safe for empty decks

Error #14

- **Failing test(s):** test_net_worth_includes_property_values, test_net_worth_uses_mortgage_value_for_mortg
- **Module:** moneypoly/player.py
- **Root cause:** Player.net_worth() returned only cash balance and ignored owned property values.
- **Fix applied:** Included owned property values (price or mortgage value when mortgaged) in net_worth().
- **Commit message used:** Error 14: Include property values in Player.net_worth

Error #15

- **Failing test(s):** test_auction_property_rejects_already_owned_property
- **Module:** moneypoly/game.py
- **Root cause:** auction_property() allowed bidding on already-owned properties.
- **Fix applied:** Added an early owner guard that skips the auction if prop.owner is set.
- **Commit message used:** Error 15: Guard auction property against already-owned tiles

Error #16

- **Failing test(s):** test_buy_property_rejects_already_owned_property
- **Module:** moneypoly/game.py
- **Root cause:** buy_property() did not guard against already-owned properties and could overwrite ownership.
- **Fix applied:** Added an early owner check that returns False if prop.owner is set.
- **Commit message used:** Error 16: Reject already-owned property purchases

Error #17

- **Failing test(s):** test_interactive_menu_loan_over_limit_does_not_crash
- **Module:** moneypoly/game.py
- **Root cause:** interactive_menu() propagated ValueError from Bank.give_loan on over-limit requests.
- **Fix applied:** Wrapped loan issuance in try/except ValueError with a user-friendly message.
- **Commit message used:** Error 17: Handle over-limit loan requests in interactive_menu

Error #18

- **Failing test(s):** test_card_collect_insufficient_bank_funds_is_safe
- **Module:** moneypoly/game.py
- **Root cause:** card_collect() propagated ValueError when bank funds were insufficient.
- **Fix applied:** Caught ValueError and treated payout as $0 with a clear message.
- **Commit message used:** Error 18: Make card_collect safe on insufficient funds

Error #19

- **Failing test(s):** test_apply_card_missing_action_is_ignored, test_apply_card_missing_value_defaults_to_
- **Module:** moneypoly/game.py
- **Root cause:** _apply_card() indexed card["action"] and card["value"] directly, raising KeyError.
- **Fix applied:** Read fields with .get(), ignore cards missing action, and default value to 0.
- **Commit message used:** Error 19: Harden apply_card against malformed cards

Error #20

- **Failing test(s):** test_main_rejects_less_than_two_players
- **Module:** moneypoly/main.py
- **Root cause:** main() started the game even when fewer than two players were provided.
- **Fix applied:** Raised ValueError when fewer than two names are entered.
- **Commit message used:** Error 20: Enforce minimum two players in main

4 Detailed White-Box Test Cases

4.1 File: test_additional_whitebox.py

Test: test_propertyspec_constructor_registers_group

- **What it checks:** Creating a Property from a PropertySpec correctly sets name/position/price/rent and registers it with a group.
- **Why needed:** Validates constructor branching for the newer PropertySpec path and ensures group linkage is created.
- **Coverage contribution:** Constructor path + group-link state update.
- **Issues found:** Error #11 (Property constructor compatibility) — fixed.

Test: test_blank_tile_returns_blank

- **What it checks:** Board.get_tile_type returns "blank" for a known blank tile index.
- **Why needed:** Confirms the default branch for tiles that are not properties or special tiles.
- **Coverage contribution:** Tile-type classification else path.
- **Issues found:** None.

Test: test_card_collect_increases_balance

- **What it checks:** card_collect adds money to the player balance.
- **Why needed:** Verifies the core path for collect-type card actions.
- **Coverage contribution:** Card action branch for collect with sufficient bank funds.
- **Issues found:** None (but related safety fixed in Error #18).

Test: test_card_pay_decreases_balance

- **What it checks:** card_pay deducts money from the player balance.
- **Why needed:** Ensures pay-type card action reduces funds, guarding financial consistency.
- **Coverage contribution:** Card action branch for pay.
- **Issues found:** None.

Test: test_card_move_to_passing_go_awards_salary

- **What it checks:** card_move_to awards salary when passing Go.
- **Why needed:** Confirms edge case of card-induced movement across Go.
- **Coverage contribution:** Move-to branch + pass-Go salary edge case.
- **Issues found:** Related to Error #4 (passing Go salary) — fixed in Player.move logic.

Test: test_collect_from_all_skips_insufficient_players

- **What it checks:** collect_from_all only collects from players who can pay.
- **Why needed:** Ensures the loop skips insufficient-balance players without crashing.
- **Coverage contribution:** Conditional branch inside collection loop.
- **Issues found:** None.

4.2 File: test_balanced_coverage_whitebox.py

Test: test_propertygroup_owner_counts_empty

- **What it checks:** Empty PropertyGroup returns empty owner count mapping.
- **Why needed:** Validates base state and avoids key errors in later counts.
- **Coverage contribution:** Empty-collection branch.
- **Issues found:** None.

Test: test_propertygroup_size_updates

- **What it checks:** PropertyGroup.size grows as properties are added.
- **Why needed:** Confirms group bookkeeping is correct for later rent calculations.
- **Coverage contribution:** Property registration and size tracking.
- **Issues found:** None.

Test: test_player_remove_absent_property_no_change

- **What it checks:** Removing a property the player doesn't own leaves counts unchanged.
- **Why needed:** Guards against negative counts and unintended side effects.
- **Coverage contribution:** Removal of non-member edge path.
- **Issues found:** None.

Test: test_bank_total_loans_multiple

- **What it checks:** Multiple loans are tracked and summed correctly.
- **Why needed:** Confirms aggregation of loan values across multiple entries.
- **Coverage contribution:** Loan list growth and total calculation.
- **Issues found:** Related to Error #6 (bank funds deduction) — fixed.

Test: test_bank_summary_prints

- **What it checks:** Bank.summary prints reserves and loans issued.
- **Why needed:** Ensures reporting path doesn't crash and contains key info.
- **Coverage contribution:** Print/report branch.
- **Issues found:** None.

Test: test_board_repr_contains_counts

- **What it checks:** Board.__repr__ includes properties count.
- **Why needed:** Confirms stable repr formatting for debugging.
- **Coverage contribution:** Representation path with non-empty board.
- **Issues found:** None.

Test: test_carddeck_repr_non_empty

- **What it checks:** CardDeck.__repr__ is valid when deck has cards.
- **Why needed:** Ensures repr is informative for non-empty decks.
- **Coverage contribution:** Repr branch with non-empty list.
- **Issues found:** None (empty-deck repr fixed in Error #13).

Test: test_carddeck_draw_then_peek

- **What it checks:** Drawing advances the deck, then peek shows next card.
- **Why needed:** Confirms draw advances index while peek does not.
- **Coverage contribution:** Draw path + peek path sequence.
- **Issues found:** None.

Test: test_safe_int_input_type_error_path

- **What it checks:** ui.safe_int_input returns default when input type is invalid (None).
- **Why needed:** Validates robustness to non-string input from mocking or broken input.
- **Coverage contribution:** Exception handling branch.
- **Issues found:** None.

Test: test_confirm_trims_spaces

- **What it checks:** ui.confirm accepts " y " with whitespace.
- **Why needed:** Ensures trimming logic handles user typing variability.
- **Coverage contribution:** Input normalization branch.
- **Issues found:** None.

Test: test_advance_turn_single_player

- **What it checks:** Game.advance_turn cycles correctly with a single player.
- **Why needed:** Confirms turn logic does not break in minimal player count.
- **Coverage contribution:** Index wrap-around edge case.
- **Issues found:** None.

Test: test_apply_card_move_to_without_passing_go_no_salary

- **What it checks:** move_to card doesn't award salary when not passing Go.
- **Why needed:** Validates correct branching between pass-Go and no-pass cases.
- **Coverage contribution:** move_to branch without salary.
- **Issues found:** None.

Test: test_apply_card_unknown_action_noop

- **What it checks:** Unknown card action leaves player state unchanged.
- **Why needed:** Ensures unexpected action strings are ignored safely.
- **Coverage contribution:** default/no-op action branch.
- **Issues found:** None (missing action/value fixed in Error #19).

Test: test_run_no_players_prints_no_players_message

- **What it checks:** Game.run prints a message when no players remain.
- **Why needed:** Verifies termination branch and user feedback.
- **Coverage contribution:** run-loop exit when players list is empty.
- **Issues found:** None.

Test: test_menu_mortgage_invalid_selection_does_not_call_action

- **What it checks:** Invalid mortgage menu selection does not call mortgage_property.
- **Why needed:** Confirms menu input validation prevents invalid actions.
- **Coverage contribution:** menu invalid-selection branch.
- **Issues found:** None.

Test: test_menu_unmortgage_invalid_selection_does_not_call_action

- **What it checks:** Invalid unmortgage selection does not call unmortgage_property.
- **Why needed:** Guards against invalid user input causing unintended changes.
- **Coverage contribution:** menu invalid-selection branch.
- **Issues found:** None.

4.3 File: test_board_bank_dice_whitebox.py

Test: test_special_tiles_mapping_has_core_positions

- **What it checks:** SPECIAL_TILES mapping includes Go, Jail, and Go-to-Jail positions.
- **Why needed:** Validates core board layout constants.
- **Coverage contribution:** Constant mapping integrity.
- **Issues found:** None.

Test: test_board_creates_expected_property_count

- **What it checks:** Board initializes with 22 properties.
- **Why needed:** Ensures board setup matches expected game design.
- **Coverage contribution:** Board initialization branch.
- **Issues found:** None.

Test: test_board_creates_expected_group_count

- **What it checks:** Board initializes with 8 property groups.
- **Why needed:** Confirms group creation path used for rent rules.
- **Coverage contribution:** Group initialization.
- **Issues found:** None.

Test: test_get_property_at_hit_and_miss

- **What it checks:** get_property_at returns property on property positions and None on non-properties.
- **Why needed:** Validates lookup behavior for both hit and miss paths.
- **Coverage contribution:** Conditional branch for property lookup.
- **Issues found:** None.

Test: test_get_tile_type_special

- **What it checks:** get_tile_type returns special tile types for tax and free parking.
- **Why needed:** Confirms special tile mapping is used in game logic.
- **Coverage contribution:** Special tile type branch.
- **Issues found:** None.

Test: test_get_tile_type_property_and_blank

- **What it checks:** get_tile_type returns "property" for property tiles and "blank" for blank tiles.
- **Why needed:** Ensures both branches are correctly distinguished.
- **Coverage contribution:** Property vs blank tile branch.
- **Issues found:** None.

Test: test_is_purchasable_false_for_non_property

- **What it checks:** is_purchasable returns False for non-property tiles.
- **Why needed:** Prevents buying non-properties.
- **Coverage contribution:** Non-property early return branch.
- **Issues found:** None.

Test: test_is_purchasable_false_for_mortgaged

- **What it checks:** mortgaged properties are not purchasable.
- **Why needed:** Enforces mortgage state logic.
- **Coverage contribution:** Mortgaged branch.
- **Issues found:** None.

Test: test_is_purchasable_false_for_owned

- **What it checks:** owned properties are not purchasable.
- **Why needed:** Ensures ownership state blocks purchase.
- **Coverage contribution:** Owned property branch.
- **Issues found:** None.

Test: test_is_purchasable_true_for_unowned_unmortgaged

- **What it checks:** unowned and unmortgaged properties are purchasable.
- **Why needed:** Confirms positive case for buying.
- **Coverage contribution:** True path for purchasable.
- **Issues found:** None.

Test: test_is_special_tile_true_false

- **What it checks:** is_special_tile returns True for Go and False for regular tiles.
- **Why needed:** Distinguishes special tile behavior in move logic.
- **Coverage contribution:** True/false branch.
- **Issues found:** None.

Test: test_owned_and_unowned_lists

- **What it checks:** properties_owned_by and unowned_properties reflect ownership state.
- **Why needed:** Ensures list-computation is correct for UI and logic.
- **Coverage contribution:** Ownership filtering branch.
- **Issues found:** None.

Test: test_initial_funds

- **What it checks:** Bank starts with configured starting funds.
- **Why needed:** Validates base state for all money flows.
- **Coverage contribution:** Constructor state.
- **Issues found:** None.

Test: test_collect_increases_funds

- **What it checks:** Bank.collect adds to funds for positive amounts.
- **Why needed:** Confirms positive collection path.
- **Coverage contribution:** Positive amount branch.
- **Issues found:** None.

Test: test_collect_negative_amount_is_ignored

- **What it checks:** Bank.collect ignores negative amounts.
- **Why needed:** Prevents exploiting bank by sending negatives.
- **Coverage contribution:** Negative guard branch.
- **Issues found:** Error #5 — fixed.

Test: test_payout_zero_or_negative_returns_zero

- **What it checks:** Bank.pay_out returns 0 for zero or negative requests.
- **Why needed:** Ensures safe behavior on invalid payout amounts.
- **Coverage contribution:** Non-positive payout branch.
- **Issues found:** None.

Test: test_payout_raises_if_insufficient

- **What it checks:** Bank.pay_out raises ValueError when funds are insufficient.
- **Why needed:** Ensures protective behavior for overdraw.
- **Coverage contribution:** Exception branch.
- **Issues found:** None.

Test: test_payout_success_decreases_funds

- **What it checks:** Bank.pay_out reduces funds on valid payout.
- **Why needed:** Confirms fund accounting for payouts.
- **Coverage contribution:** Successful payout path.
- **Issues found:** None.

Test: test_give_loan_non_positive_ignored

- **What it checks:** give_loan ignores non-positive loan amounts.
- **Why needed:** Prevents nonsensical or negative loans.
- **Coverage contribution:** Non-positive guard branch.
- **Issues found:** None.

Test: test_give_loan_positive_tracks

- **What it checks:** give_loan increases player balance and tracks loan count.
- **Why needed:** Confirms loan accounting and player balance updates.
- **Coverage contribution:** Loan issuance branch.
- **Issues found:** Error #6 — fixed.

Test: test_reset_clears_values

- **What it checks:** Dice.reset clears die values and doubles streak.
- **Why needed:** Validates reset state for new turn.
- **Coverage contribution:** State reset path.
- **Issues found:** None.

Test: test_roll_calls_randint_with_full_six_sided_range

- **What it checks:** Dice.roll calls randint with range (1, 6) for both dice.
- **Why needed:** Ensures correct dice distribution including value 6.
- **Coverage contribution:** External RNG call parameters.
- **Issues found:** Error #2 — fixed.

Test: test_roll_updates_streak_on_doubles

- **What it checks:** doubles streak increments when roll is doubles.
- **Why needed:** Supports jail-on-three-doubles rule.
- **Coverage contribution:** Doubles branch.
- **Issues found:** None.

Test: test_roll_resets_streak_on_non_doubles

- **What it checks:** doubles streak resets after non-doubles roll.
- **Why needed:** Prevents incorrect accumulation.
- **Coverage contribution:** Non-doubles branch.
- **Issues found:** None.

Test: test_is_doubles_total_describe

- **What it checks:** is_doubles, total, and describe behave consistently.
- **Why needed:** Ensures dice reporting and state are coherent.
- **Coverage contribution:** Utility method branches.
- **Issues found:** None.

4.4 File: test_cards_whitebox.py

Test: test_chance_cards_have_required_keys

- **What it checks:** All Chance cards include description, action, and value.
- **Why needed:** Prevents malformed card data from crashing game logic.
- **Coverage contribution:** Card data validation loop.
- **Issues found:** None.

Test: test_community_cards_have_required_keys

- **What it checks:** All Community Chest cards include description, action, and value.
- **Why needed:** Same as chance cards for the second deck.
- **Coverage contribution:** Card data validation loop.
- **Issues found:** None.

Test: test_card_actions_are_known_set

- **What it checks:** Card action values are in the allowed action set.
- **Why needed:** Protects against invalid actions causing undefined behavior.
- **Coverage contribution:** Enum validation branch.
- **Issues found:** None.

Test: test_len_matches_input_cards

- **What it checks:** CardDeck.__len__ equals number of input cards.
- **Why needed:** Validates deck size tracking.
- **Coverage contribution:** Length method.
- **Issues found:** None.

Test: test_draw_none_when_empty

- **What it checks:** draw returns None on empty deck.
- **Why needed:** Prevents exceptions when deck is empty.
- **Coverage contribution:** Empty-deck draw branch.
- **Issues found:** None.

Test: test_peek_none_when_empty

- **What it checks:** peek returns None on empty deck.
- **Why needed:** Ensures peek is safe on empty deck.
- **Coverage contribution:** Empty-deck peek branch.
- **Issues found:** None.

Test: test_cards_remaining_empty_expected_zero

- **What it checks:** cards_remaining returns 0 on empty deck.
- **Why needed:** Guards against division-by-zero and incorrect counts.
- **Coverage contribution:** Empty-deck count branch.
- **Issues found:** Error #1 — fixed.

Test: test_peek_does_not_advance

- **What it checks:** peek does not move the internal index.
- **Why needed:** Ensures peek is non-destructive.
- **Coverage contribution:** Peek branch with index stability.
- **Issues found:** None.

Test: test_draw_advances

- **What it checks:** draw advances to the next card.
- **Why needed:** Confirms draw progression.
- **Coverage contribution:** Draw sequence branch.
- **Issues found:** None.

Test: test_draw_cycles_when_exhausted

- **What it checks:** draw cycles back to the start when deck is exhausted.
- **Why needed:** Ensures deck wraps without errors.
- **Coverage contribution:** Modulo/wrap branch.
- **Issues found:** None.

Test: test_cards_remaining_counts_down_and_wraps

- **What it checks:** cards_remaining decreases as cards are drawn and wraps after full cycle.
- **Why needed:** Confirms count logic across cycles.
- **Coverage contribution:** Counter update + wrap branch.
- **Issues found:** None.

Test: test_reshuffle_resets_index

- **What it checks:** reshuffle resets index to start and allows full count.
- **Why needed:** Ensures reshuffle restores full deck visibility.
- **Coverage contribution:** Reshuffle path.
- **Issues found:** None.

4.5 File: test_exhaustive_coverage_whitebox.py

Test: test_config_constants_sanity

- **What it checks:** key config constants are positive and in valid ranges.
- **Why needed:** Prevents impossible board positions or negative money amounts.
- **Coverage contribution:** Constant validation branch.
- **Issues found:** None.

Test: test_repr_methods_are_stable_for_non_empty_objects

- **What it checks:** repr outputs for Player, Property, PropertyGroup, Bank, Board, CardDeck contain class names.
- **Why needed:** Ensures debug-friendly representations across models.
- **Coverage contribution:** Representation branches for multiple types.
- **Issues found:** None.

Test: test_cards_remaining_after_multiple_cycles

- **What it checks:** cards remaining returns correct value after multiple cycles.
- **Why needed:** Confirms long-run deck stability.
- **Coverage contribution:** Long-run modulo path.
- **Issues found:** None.

Test: test_draw_empty_then_reshuffle_stays_safe

- **What it checks:** draw and reshuffle on empty deck remain safe and return None.
- **Why needed:** Ensures empty deck never crashes.
- **Coverage contribution:** Empty-deck + reshuffle branch.
- **Issues found:** None.

Test: test_move_and_resolve_blank_tile_only_checks_bankruptcy

- **What it checks:** blank tile triggers only bankruptcy check.
- **Why needed:** Confirms no side effects on blank tiles.
- **Coverage contribution:** Blank-tile branch in move resolution.
- **Issues found:** None.

Test: test_move_and_resolve_chance_with_none_card

- **What it checks:** chance tile with no card still calls _apply_card(None).
- **Why needed:** Ensures consistent flow even when deck is empty.
- **Coverage contribution:** Chance branch + None card.
- **Issues found:** None.

Test: test_apply_card_move_to_property_tile_but_no_property_object

- **What it checks:** move to on property tile skips handler if property object is missing.
- **Why needed:** Avoids None dereference and incorrect call.
- **Coverage contribution:** Guard branch for missing property.
- **Issues found:** None.

Test: test_apply_card_birthday_does_not_charge_current_player

- **What it checks:** birthday action does not take money from current player.
- **Why needed:** Ensures correct payer set in multi-player transfers.
- **Coverage contribution:** Loop with skip-current-player branch.
- **Issues found:** None.

Test: test_apply_card_collect_from_all_does_not_charge_current_player

- **What it checks:** collect from all does not charge the current player.
- **Why needed:** Prevents self-charging bug.
- **Coverage contribution:** Loop skip branch.
- **Issues found:** None.

Test: test_interactive_menu_ignores_unknown_choice

- **What it checks:** unknown menu choice is ignored safely.
- **Why needed:** Prevents crashes on unexpected input.
- **Coverage contribution:** Default case branch in menu.
- **Issues found:** None.

Test: test_check_bankruptcy_when_last_player_removed

- **What it checks:** removing last player clears list and resets index.
- **Why needed:** Ensures game does not keep invalid index.
- **Coverage contribution:** Last-player removal branch.
- **Issues found:** None.

Test: test_run_exits_when_running_flag_false

- **What it checks:** run exits immediately if running flag is False.
- **Why needed:** Confirms termination guard works.
- **Coverage contribution:** Early-exit branch in run.
- **Issues found:** None.

Test: test_menu_trade_negative_cash_still_routes_to_trade

- **What it checks:** trade menu passes negative cash value through.
- **Why needed:** Covers edge case and ensures routing does not crash.
- **Coverage contribution:** Trade menu routing branch.
- **Issues found:** None (business rule may be refined later).

Test: test_auction_rejects_equal_to_current_high_without_min_increment

- **What it checks:** auction rejects bids that do not meet min increment.
- **Why needed:** Ensures auction rules enforce minimum raise.
- **Coverage contribution:** Min-increment branch.
- **Issues found:** None.

Test: test_menu_mortgage_handles_default_zero_selection

- **What it checks:** selection 0 exits mortgage menu without action.
- **Why needed:** Validates safe default handling.
- **Coverage contribution:** Zero selection branch.
- **Issues found:** None.

Test: test_menu_unmortgage_handles_default_zero_selection

- **What it checks:** selection 0 exits unmortgage menu without action.
- **Why needed:** Validates safe default handling.
- **Coverage contribution:** Zero selection branch.
- **Issues found:** None.

4.6 File: test_game_core_whitebox.py

Test: test_current_player_and_advance

- **What it checks:** current player returns correct player and advance turn updates index.
- **Why needed:** Ensures turn order correctness.
- **Coverage contribution:** Basic turn sequencing.
- **Issues found:** None.

Test: test_play_turn_jailed_player_uses_jail_handler_and_advances

- **What it checks:** jailed player uses jail handler and turn advances.
- **Why needed:** Confirms jail branch of play_turn.
- **Coverage contribution:** Jailed player path.
- **Issues found:** None.

Test: test_play_turn_three_doubles_sends_to_jail

- **What it checks:** three doubles sends player to jail and advances index.
- **Why needed:** Enforces core Monopoly rule.
- **Coverage contribution:** Doubles-streak branch.
- **Issues found:** None.

Test: test_play_turn_doubles_extra_turn_no_advance

- **What it checks:** doubles grant extra turn (no index advance).
- **Why needed:** Ensures correct handling for doubles.
- **Coverage contribution:** Doubles branch (extra turn).
- **Issues found:** None.

Test: test_play_turn_non_doubles_advances

- **What it checks:** non-doubles advance to next player.
- **Why needed:** Confirms normal turn progression.
- **Coverage contribution:** Non-doubles branch.
- **Issues found:** None.

Test: test_check_bankruptcy_non_bankrupt_does_nothing

- **What it checks:** non-bankrupt player remains in game.
- **Why needed:** Prevents accidental elimination.
- **Coverage contribution:** Bankruptcy guard branch.
- **Issues found:** None.

Test: test_check_bankruptcy_removes_player_and_resets_props

- **What it checks:** bankrupt player is removed, properties reset, and eliminated flag set.
- **Why needed:** Ensures cleanup logic on bankruptcy.
- **Coverage contribution:** Bankruptcy removal branch.
- **Issues found:** None.

Test: test_check_bankruptcy_resets_current_index_when_out_of_range

- **What it checks:** current index resets if it becomes invalid after removal.
- **Why needed:** Prevents index errors after player removal.
- **Coverage contribution:** Index correction branch.
- **Issues found:** None.

Test: test_find_winner_none_when_no_players

- **What it checks:** find_winner returns None when players list is empty.
- **Why needed:** Prevents errors in end-game logic.
- **Coverage contribution:** Empty list branch.
- **Issues found:** None.

Test: test_find_winner_should_pick_richest

- **What it checks:** find_winner chooses highest net worth player.
- **Why needed:** Ensures winner selection logic is correct.
- **Coverage contribution:** Winner selection branch.
- **Issues found:** Error #3 — fixed.

Test: test_run_breaks_immediately_with_single_player

- **What it checks:** game.run does not loop when only one player exists.
- **Why needed:** Prevents unnecessary turns for finished game.
- **Coverage contribution:** Early termination branch.
- **Issues found:** None.

4.7 File: test_game_tiles_cards_whitebox.py

Test: test_move_resolve_income_tax

- **What it checks:** income tax reduces player balance and increases bank funds.
- **Why needed:** Verifies tax branch updates both entities correctly.
- **Coverage contribution:** Income tax branch.
- **Issues found:** None.

Test: test_move_resolve_luxury_tax

- **What it checks:** luxury tax reduces player balance and increases bank funds.
- **Why needed:** Confirms luxury tax branch behavior.
- **Coverage contribution:** Luxury tax branch.
- **Issues found:** None.

Test: test_move_resolve_go_to_jail

- **What it checks:** go to jail tile sets in jail and positions player to jail.
- **Why needed:** Ensures special tile enforcement.
- **Coverage contribution:** Go-to-jail branch.
- **Issues found:** None.

Test: test_move_resolve_free_parking_no_money_change

- **What it checks:** free parking does not change balance.
- **Why needed:** Confirms no unintended money movement.
- **Coverage contribution:** Free parking branch.
- **Issues found:** None.

Test: test_move_resolve_chance_draws_card

- **What it checks:** chance tile draws from chance deck and applies card.
- **Why needed:** Ensures card draw pipeline on chance tiles.
- **Coverage contribution:** Chance tile branch.
- **Issues found:** Error #12 — fixed.

Test: test_move_resolve_community_draws_card

- **What it checks:** community chest tile draws from community deck and applies card.
- **Why needed:** Ensures card draw pipeline on community chest tiles.
- **Coverage contribution:** Community chest branch.
- **Issues found:** Error #12 — fixed.

Test: test_move_resolve_property_calls_property_handler_when_prop_exists

- **What it checks:** property tile calls handle_property_tile when property exists.
- **Why needed:** Ensures correct property handling flow.
- **Coverage contribution:** Property tile branch.
- **Issues found:** None.

Test: test_move_resolve_property_skips_when_no_prop

- **What it checks:** missing property object skips handler call.
- **Why needed:** Avoids None dereference on corrupted board state.
- **Coverage contribution:** Guard branch.
- **Issues found:** None.

Test: test_move_resolve_railroad_calls_property_handler

- **What it checks:** railroad tile uses property handler.
- **Why needed:** Ensures railroads are handled as properties.
- **Coverage contribution:** Railroad branch.
- **Issues found:** None.

Test: test_apply_card_none_noop

- **What it checks:** apply_card does nothing when card is None.
- **Why needed:** Guards against empty decks or draw returning None.
- **Coverage contribution:** None-card branch.
- **Issues found:** None.

Test: test_apply_card_collect

- **What it checks:** collect action increases player balance.
- **Why needed:** Validates collect path in card handler.
- **Coverage contribution:** collect action branch.
- **Issues found:** None.

Test: test_apply_card_pay

- **What it checks:** pay action reduces player balance.
- **Why needed:** Validates pay branch in card handler.
- **Coverage contribution:** pay action branch.
- **Issues found:** None.

Test: test_apply_card_jail

- **What it checks:** jail action sets in_jail True.
- **Why needed:** Ensures jail card behavior.
- **Coverage contribution:** jail action branch.
- **Issues found:** None.

Test: test_apply_card_jail_free

- **What it checks:** jail free action increments get_out_of_jail_cards.
- **Why needed:** Ensures card grants correct resource.
- **Coverage contribution:** jail_free branch.
- **Issues found:** None.

Test: test_apply_card_move_to_passing_go_gives_salary

- **What it checks:** move to action awards salary when passing Go.
- **Why needed:** Ensures pass-Go logic in card movement.
- **Coverage contribution:** move_to with salary branch.
- **Issues found:** Related to Error #4 (pass-Go salary) — fixed.

Test: test_apply_card_move_to_property_invokes_property_handler

- **What it checks:** move to property calls property handler.
- **Why needed:** Ensures property resolution after card movement.
- **Coverage contribution:** move_to + property branch.
- **Issues found:** None.

Test: test_apply_card_birthday_only_collects_from_players_who_can_pay

- **What it checks:** birthday action only collects from players who can pay.
- **Why needed:** Prevents negative balances from forced payments.
- **Coverage contribution:** conditional collection branch.
- **Issues found:** None.

Test: test_apply_card_collect_from_all_only_collects_from_players_who_can_pay

- **What it checks:** collect_from_all skips players with insufficient funds.
- **Why needed:** Ensures fair handling and avoids crashes.
- **Coverage contribution:** conditional collection branch.
- **Issues found:** None.

Test: test_jail_turn_use_card_yes

- **What it checks:** player uses get-out-of-jail card and moves.
- **Why needed:** Validates jail card branch with confirm True.
- **Coverage contribution:** jail card branch.
- **Issues found:** None.

Test: test_jail_turn_card_no_then_pay_yes

- **What it checks:** player declines card, pays fine, and moves.
- **Why needed:** Covers confirm False then True path.
- **Coverage contribution:** jail fine payment branch.
- **Issues found:** Related to Error #9 (fine deducted) — fixed.

Test: test_jail_turn_no_action_under_three_turns

- **What it checks:** player stays in jail and increments jail_turns under 3.
- **Why needed:** Covers waiting path without payment.
- **Coverage contribution:** jail wait branch.
- **Issues found:** None.

Test: test_jail_turn_mandatory_release_on_third_turn

- **What it checks:** player is released and fined on third turn.
- **Why needed:** Confirms forced payment branch.
- **Coverage contribution:** third-turn release branch.
- **Issues found:** None (fine deduction fixed in Error #9).

4.8 File: test_game_transactions_menus_whitebox.py

Test: test_handle_property_tile_buy_path

- **What it checks:** buying property via input "b" assigns owner.
- **Why needed:** Validates buy path in property handling.
- **Coverage contribution:** buy branch.
- **Issues found:** None.

Test: test_handle_property_tile_auction_path

- **What it checks:** input "a" triggers auction_property.
- **Why needed:** Ensures auction branch is wired correctly.
- **Coverage contribution:** auction branch.
- **Issues found:** None.

Test: test_handle_property_tile_skip_path

- **What it checks:** input "s" skips buy/auction actions.
- **Why needed:** Confirms skip branch does not alter ownership.
- **Coverage contribution:** skip branch.
- **Issues found:** None.

Test: test_handle_property_tile_owned_by_self_no_rent

- **What it checks:** landing on own property does not pay rent.
- **Why needed:** Prevents self-payment bug.
- **Coverage contribution:** owned-by-self branch.
- **Issues found:** None.

Test: test_handle_property_tile_owned_by_other_pays_rent

- **What it checks:** landing on others' property calls pay_rent.
- **Why needed:** Ensures rent path is invoked.
- **Coverage contribution:** owned-by-other branch.
- **Issues found:** Related to Error #8 (owner credited) — fixed.

Test: test_buy_property_success_and_boundary_failure

- **What it checks:** buy_property succeeds with sufficient funds and fails when underfunded.
- **Why needed:** Verifies affordability boundary logic.
- **Coverage contribution:** success and failure branches.
- **Issues found:** Related to Error #7 (exact balance) — fixed.

Test: test_pay_rent_mortgaged_unowned_owned

- **What it checks:** rent is zero for unowned/mortgaged and charged for owned.
- **Why needed:** Confirms rent logic across property states.
- **Coverage contribution:** rent branch across ownership/mortgage states.
- **Issues found:** Related to Error #8 — fixed.

Test: test_mortgage_property_branches

- **What it checks:** mortgage behavior for non-owner, owner first time, repeat, and other owner.
- **Why needed:** Validates mortgage permissions and idempotence.
- **Coverage contribution:** multiple mortgage branches.
- **Issues found:** None.

Test: test_unmortgage_property_branches

- **What it checks:** unmortgage fails when not owner, not mortgaged, insufficient funds, and succeeds with funds.
- **Why needed:** Ensures all unmortgage checks are enforced.
- **Coverage contribution:** multiple unmortgage branches.
- **Issues found:** None.

Test: test_trade_success_and_failures

- **What it checks:** trade fails when property not owned or buyer lacks funds, succeeds otherwise.
- **Why needed:** Confirms trade preconditions and successful transfer.
- **Coverage contribution:** trade failure and success branches.
- **Issues found:** Related to Error #10 — fixed.

Test: test_auction_no_bids

- **What it checks:** auction ends with no owner when all bids are zero.
- **Why needed:** Ensures auction handles no-bid path.
- **Coverage contribution:** no-bid branch.
- **Issues found:** None.

Test: test_auction_highest_valid_bidder_wins

- **What it checks:** auction assigns property to highest valid bidder.
- **Why needed:** Confirms auction winner selection and filtering by balance.
- **Coverage contribution:** highest-bid selection branch.
- **Issues found:** None.

Test: test_interactive_menu_exits_on_zero

- **What it checks:** menu exits when user selects 0.
- **Why needed:** Validates immediate exit path.
- **Coverage contribution:** exit branch.
- **Issues found:** None.

Test: test_interactive_menu_routes_to_view_standings

- **What it checks:** menu option 1 calls print_standings.
- **Why needed:** Ensures menu routing to standings works.
- **Coverage contribution:** menu route branch.
- **Issues found:** None.

Test: test_interactive_menu_routes_to_board_ownership

- **What it checks:** menu option 2 calls print_board_ownership.
- **Why needed:** Ensures menu routing to board ownership works.
- **Coverage contribution:** menu route branch.
- **Issues found:** None.

Test: test_interactive_menu_routes_to_submenus_and_loan

- **What it checks:** menu routes to mortgage, unmortgage, trade, and loan, with loan called once.
- **Why needed:** Confirms multi-option routing logic.
- **Coverage contribution:** submenu routing branch.
- **Issues found:** Related to Error #17 — fixed.

Test: test_menu_mortgage_no_candidates

- **What it checks:** mortgage menu handles case with no eligible properties.
- **Why needed:** Prevents crashes on empty lists.
- **Coverage contribution:** empty-candidates branch.
- **Issues found:** None.

Test: test_menu_mortgage_selected

- **What it checks:** selecting a property triggers mortgage_property.
- **Why needed:** Ensures selection mapping to action.
- **Coverage contribution:** selection branch.
- **Issues found:** None.

Test: test_menu_unmortgage_no_candidates

- **What it checks:** unmortgage menu handles no mortgaged properties.
- **Why needed:** Prevents errors on empty list.
- **Coverage contribution:** empty-candidates branch.
- **Issues found:** None.

Test: test_menu_unmortgage_selected

- **What it checks:** selecting a mortgaged property triggers unmortgage_property.
- **Why needed:** Ensures selection branch works.
- **Coverage contribution:** selection branch.
- **Issues found:** None.

Test: test_menu_trade_no_other_players

- **What it checks:** trade menu handles single-player case.
- **Why needed:** Prevents invalid partner selection.
- **Coverage contribution:** no-partner branch.
- **Issues found:** None.

Test: test_menu_trade_invalid_partner_index

- **What it checks:** invalid partner index is handled without trade.
- **Why needed:** Validates input bounds in trade menu.
- **Coverage contribution:** invalid-index branch.
- **Issues found:** None.

Test: test_menu_trade_no_properties

- **What it checks:** trade menu handles player with no properties.
- **Why needed:** Prevents selection from empty list.
- **Coverage contribution:** no-properties branch.
- **Issues found:** None.

Test: test_menu_trade_invalid_property_index

- **What it checks:** invalid property index is handled safely.
- **Why needed:** Validates property selection bounds.
- **Coverage contribution:** invalid-index branch.
- **Issues found:** None.

Test: test_menu_trade_success_path

- **What it checks:** valid trade selection calls trade with correct parameters.
- **Why needed:** Confirms positive trade route.
- **Coverage contribution:** success branch.
- **Issues found:** Related to Error #10 — fixed.

4.9 File: test_main_ui_whitebox.py

Test: test_get_player_names_splits_and_strips

- **What it checks:** input names are split and stripped of spaces.
- **Why needed:** Prevents unexpected whitespace in player names.
- **Coverage contribution:** parsing branch.
- **Issues found:** None.

Test: test_get_player_names_ignores_empty_tokens

- **What it checks:** empty tokens are dropped from names list.
- **Why needed:** Ensures valid player list for game setup.
- **Coverage contribution:** filtering branch.
- **Issues found:** None.

Test: test_main_keyboard_interrupt_path

- **What it checks:** KeyboardInterrupt during game.run prints a friendly message.
- **Why needed:** Ensures graceful interruption handling.
- **Coverage contribution:** exception handling branch.
- **Issues found:** None.

Test: test_main_value_error_path

- **What it checks:** ValueError during setup prints a setup error message.
- **Why needed:** Ensures errors are user-visible.
- **Coverage contribution:** exception handling branch.
- **Issues found:** None.

Test: test_format_currency

- **What it checks:** ui.format_currency formats integers with commas and dollars.
- **Why needed:** Ensures consistent output formatting.
- **Coverage contribution:** utility formatting branch.
- **Issues found:** None.

Test: test_safe_int_input_valid

- **What it checks:** valid integer input returns integer.
- **Why needed:** Confirms happy path for user input.
- **Coverage contribution:** valid input branch.
- **Issues found:** None.

Test: test_safe_int_input_invalid_uses_default

- **What it checks:** invalid input returns provided default.
- **Why needed:** Ensures safe fallback path.
- **Coverage contribution:** invalid input branch.
- **Issues found:** None.

Test: test_confirm_true_for_y_only

- **What it checks:** confirm returns True for "y" or "Y" only.
- **Why needed:** Enforces strict confirm semantics.
- **Coverage contribution:** confirm parsing branch.
- **Issues found:** None.

Test: test_print_banner_outputs_title

- **What it checks:** print_banner prints the given title.
- **Why needed:** Ensures UI output includes header.
- **Coverage contribution:** output branch.
- **Issues found:** None.

Test: test_print_player_card_with_and_without_properties

- **What it checks:** player card prints "none" when empty and lists properties when present.
- **Why needed:** Confirms output differs for empty vs non-empty state.
- **Coverage contribution:** two output state branches.
- **Issues found:** None.

Test: test_print_standings_sorts_by_net_worth

- **What it checks:** standings output orders players by net worth.
- **Why needed:** Ensures sorting logic matches spec.
- **Coverage contribution:** sort branch.
- **Issues found:** None (related to Error #14 net worth correctness).

Test: test_print_board_ownership_shows_mortgage_flag

- **What it checks:** board ownership output shows mortgaged flag.
- **Why needed:** Confirms UI shows key property state.
- **Coverage contribution:** UI branch for mortgaged property.
- **Issues found:** None.

4.10 File: test_models_whitebox.py

Test: test_player_defaults

- **What it checks:** Player initializes with default name, position, properties, jail state, and elimination flag.
- **Why needed:** Establishes baseline state for all later logic.
- **Coverage contribution:** Player constructor defaults.
- **Issues found:** None.

Test: test_add_money_positive

- **What it checks:** add_money increases balance with positive amount.
- **Why needed:** Confirms normal money addition behavior.
- **Coverage contribution:** positive amount branch.
- **Issues found:** None.

Test: test_add_money_negative_raises

- **What it checks:** add_money raises ValueError on negative input.
- **Why needed:** Prevents invalid balance manipulation.
- **Coverage contribution:** error branch.
- **Issues found:** None.

Test: test_deduct_money_positive

- **What it checks:** deduct_money decreases balance with positive amount.
- **Why needed:** Confirms normal deduction path.
- **Coverage contribution:** positive deduction branch.
- **Issues found:** None.

Test: test_deduct_money_negative_raises

- **What it checks:** deduct_money raises ValueError on negative input.
- **Why needed:** Guards against invalid deduction.
- **Coverage contribution:** error branch.
- **Issues found:** None.

Test: test_is_bankrupt_true_at_zero

- **What it checks:** is_bankrupt is True when balance is zero.
- **Why needed:** Ensures bankruptcy threshold is enforced.
- **Coverage contribution:** boundary branch at zero.
- **Issues found:** None.

Test: test_is_bankrupt_false_above_zero

- **What it checks:** is_bankrupt is False when balance > 0.
- **Why needed:** Confirms normal non-bankrupt state.
- **Coverage contribution:** positive-balance branch.
- **Issues found:** None.

Test: test_net_worth_matches_balance

- **What it checks:** net_worth equals balance when no properties exist.
- **Why needed:** Verifies base-case net worth.
- **Coverage contribution:** net_worth base branch.
- **Issues found:** None (property-inclusive net worth fixed in Error #14).

Test: test_move_wraps_board

- **What it checks:** move wraps around board when passing position 39.
- **Why needed:** Confirms modulo movement behavior.
- **Coverage contribution:** wrap-around branch.
- **Issues found:** None.

Test: test_move_landing_on_go_collects_salary

- **What it checks:** move to Go awards salary.
- **Why needed:** Confirms landing-on-Go salary case.
- **Coverage contribution:** landing-on-Go branch.
- **Issues found:** None (passing-Go fixed in Error #4).

Test: test_go_to_jail_updates_all_jail_state

- **What it checks:** go_to_jail sets position, in_jail, and resets jail_turns.
- **Why needed:** Ensures jail transition updates all state fields.
- **Coverage contribution:** jail state transition branch.
- **Issues found:** None.

Test: test_add_remove_property_idempotent

- **What it checks:** adding same property twice does not duplicate; removing twice is safe.
- **Why needed:** Prevents duplicate entries and negative counts.
- **Coverage contribution:** idempotence branch.
- **Issues found:** None.

Test: test_status_line_contains_jail_tag_when_jailed

- **What it checks:** status line shows [JAILED] when in jail.
- **Why needed:** Ensures UI state reflects jail status.
- **Coverage contribution:** jail tag branch.
- **Issues found:** None.

Test: test_status_line_omits_jail_tag_when_free

- **What it checks:** status line does not show [JAILED] when not in jail.
- **Why needed:** Avoids incorrect UI flags.
- **Coverage contribution:** non-jail branch.
- **Issues found:** None.

Test: test_property_defaults

- **What it checks:** Property defaults for owner, mortgage, and mortgage value.
- **Why needed:** Ensures correct default property state.
- **Coverage contribution:** Property constructor defaults.
- **Issues found:** Error #11 — fixed.

Test: test_group_registration_happens_in_constructor

- **What it checks:** Property registers with its PropertyGroup during construction.
- **Why needed:** Ensures group membership for rent calculations.
- **Coverage contribution:** constructor group-link branch.
- **Issues found:** Error #11 — fixed.

Test: test_get_rent_base_no_group

- **What it checks:** get rent returns base rent when no group bonus applies.
- **Why needed:** Verifies base rent path.
- **Coverage contribution:** base rent branch.
- **Issues found:** None.

Test: test_get_rent_zero_when_mortgaged

- **What it checks:** mortgaged properties return zero rent.
- **Why needed:** Enforces mortgage rules.
- **Coverage contribution:** mortgaged branch.
- **Issues found:** None.

Test: test_get_rent_doubles_with_full_group_owned

- **What it checks:** rent doubles when a single owner owns the full group.
- **Why needed:** Ensures group bonus logic.
- **Coverage contribution:** full-group bonus branch.
- **Issues found:** None.

Test: test_get_rent_not_doubled_with_partial_group

- **What it checks:** rent does not double when group ownership is partial.
- **Why needed:** Prevents accidental bonus.
- **Coverage contribution:** partial-group branch.
- **Issues found:** None.

Test: test_mortgage_success_then_repeat_returns_zero

- **What it checks:** mortgage returns value once and 0 on repeat.
- **Why needed:** Enforces idempotent mortgage behavior.
- **Coverage contribution:** mortgage idempotence branch.
- **Issues found:** None.

Test: test_unmortgage_when_not_mortgaged_returns_zero

- **What it checks:** unmortgage returns 0 when not mortgaged.
- **Why needed:** Safe no-op behavior.
- **Coverage contribution:** unmortgage guard branch.
- **Issues found:** None.

Test: test_unmortgage_success_clears_flag

- **What it checks:** unmortgage clears mortgage flag and returns cost.
- **Why needed:** Confirms state reset on success.
- **Coverage contribution:** successful unmortgage branch.
- **Issues found:** None.

Test: test_is_available_true_only_unowned_unmortgaged

- **What it checks:** is_available is True only when property is unowned and unmortgaged.
- **Why needed:** Prevents buying unavailable properties.
- **Coverage contribution:** availability branch.
- **Issues found:** None.

Test: test_group_all_owned_by_false_for_none

- **What it checks:** all_owned_by returns False for None owner.
- **Why needed:** Ensures ownership checks are strict.
- **Coverage contribution:** None owner branch.
- **Issues found:** None.

Test: test_group_all_owned_by_true_when_all_match

- **What it checks:** all_owned_by True when all properties share same owner.
- **Why needed:** Enables rent bonus logic.
- **Coverage contribution:** full ownership branch.
- **Issues found:** None.

Test: test_group_add_property_backlinks_and_prevents_duplicates

- **What it checks:** add_property sets group backlink and prevents duplicates.
- **Why needed:** Maintains consistent group data.
- **Coverage contribution:** add/duplicate prevention branch.
- **Issues found:** None.

Test: test_group_owner_counts_only_counts_non_none

- **What it checks:** owner count mapping excludes None owners.
- **Why needed:** Prevents meaningless counts.
- **Coverage contribution:** filter branch.
- **Issues found:** None.

4.11 File: test_reference_report_whitebox.py

Test: test_net_worth_includes_property_values

- **What it checks:** net worth includes owned property price.
- **Why needed:** Ensures net worth reflects assets.
- **Coverage contribution:** asset inclusion branch.
- **Issues found:** Error #14 — fixed.

Test: test_net_worth_uses_mortgage_value_for_mortgaged

- **What it checks:** net worth uses mortgage value for mortgaged property.
- **Why needed:** Confirms reduced asset value when mortgaged.
- **Coverage contribution:** mortgaged asset valuation branch.
- **Issues found:** Error #14 — fixed.

Test: test_buy_property_rejects_already_owned_property

- **What it checks:** buy property returns False when property already has an owner.
- **Why needed:** Prevents ownership overwrite.
- **Coverage contribution:** owned-property guard branch.
- **Issues found:** Error #16 — fixed.

Test: test_auction_property_rejects_already_owned_property

- **What it checks:** auction property does not proceed when property is already owned.
- **Why needed:** Prevents invalid auctions.
- **Coverage contribution:** owner-guard branch.
- **Issues found:** Error #15 — fixed.

Test: test_interactive_menu_loan_over_limit_does_not_crash

- **What it checks:** loan requests over bank funds do not crash menu.
- **Why needed:** Ensures UI resilience to over-limit loans.
- **Coverage contribution:** exception handling branch.
- **Issues found:** Error #17 — fixed.

Test: test_card_collect_insufficient_bank_funds_is_safe

- **What it checks:** card collect does not crash when bank funds are insufficient.
- **Why needed:** Prevents runtime error in low-funds scenario.
- **Coverage contribution:** exception handling branch for payouts.
- **Issues found:** Error #18 — fixed.

Test: test_apply_card_missing_action_is_ignored

- **What it checks:** apply card ignores cards missing action.
- **Why needed:** Protects against malformed card data.
- **Coverage contribution:** missing-action guard branch.
- **Issues found:** Error #19 — fixed.

Test: test_apply_card_missing_value_defaults_to_zero

- **What it checks:** apply card defaults missing value to zero.
- **Why needed:** Prevents KeyError and allows safe no-op effects.
- **Coverage contribution:** missing-value default branch.
- **Issues found:** Error #19 — fixed.

Test: test_carddeck_repr_safe_when_empty

- **What it checks:** CardDeck.__repr__ returns safe string for empty deck.
- **Why needed:** Prevents crash on empty deck repr.
- **Coverage contribution:** empty repr branch.
- **Issues found:** Error #13 — fixed.

Test: test_main_rejects_less_than_two_players

- **What it checks:** main rejects setups with fewer than two players.
- **Why needed:** Enforces minimum valid game setup.
- **Coverage contribution:** setup validation branch.
- **Issues found:** Error #20 — fixed.

4.12 File: test_strict_rules_whitebox.py

Test: test_player_move_passing_go_collects_salary

- **What it checks:** player gets salary when passing Go (not just landing).
- **Why needed:** Ensures rule compliance with docstrings.
- **Coverage contribution:** pass-Go branch.
- **Issues found:** Error #4 — fixed.

Test: test_bank_collect_negative_amount_is_ignored

- **What it checks:** bank ignores negative collection.
- **Why needed:** Enforces docstring behavior and prevents negative credit.
- **Coverage contribution:** negative guard branch.
- **Issues found:** Error #5 — fixed.

Test: test_bank_give_loan_reduces_bank_funds

- **What it checks:** give loan reduces bank funds.
- **Why needed:** Enforces correct bank accounting.
- **Coverage contribution:** loan issue branch.
- **Issues found:** Error #6 — fixed.

Test: test_buy_property_with_exact_balance_should_succeed

- **What it checks:** buy succeeds when balance equals price.
- **Why needed:** Ensures boundary condition is inclusive.
- **Coverage contribution:** exact-balance branch.
- **Issues found:** Error #7 — fixed.

Test: test_pay_rent_transfers_money_to_owner

- **What it checks:** rent payment credits owner balance.
- **Why needed:** Ensures money transfer is complete and fair.
- **Coverage contribution:** rent transfer branch.
- **Issues found:** Error #8 — fixed.

Test: test_jail_pay_fine_should_deduct_player_balance

- **What it checks:** paying fine removes money from player balance.
- **Why needed:** Ensures correct accounting in jail release path.
- **Coverage contribution:** voluntary fine branch.
- **Issues found:** Error #9 — fixed.

Test: test_trade_should_credit_seller_cash

- **What it checks:** trade credits cash to seller as well as debiting buyer.
- **Why needed:** Ensures cash transfer symmetry in trades.
- **Coverage contribution:** trade success branch.
- **Issues found:** Error #10 — fixed.

5 Continued Coverage Summaries

File: test_game_tiles_cards_whitebox.py
This file also ensures tile resolution logic, card actions, and jail logic cover all major branches.

File: test_game_transactions_menus_whitebox.py
This file confirms property transactions, auctions, and interactive menus handle all user selections and edge inputs.

File: test_main_ui_whitebox.py
This file covers main entry parsing, exception handling, and UI formatting functions.